Surrogate Modelling from First Principles

A private working textbook

Written for J. Poznanski

Draft, October 12, 2026

How to read this book

This is a companion to the surrogate-modelling paper, not part of it. The paper's review says *who* did *what*; this book says *why it works*, *what it is really computing*, and *when it must fail*. Every tool is built up from things you already know, and every named property is restated in words together with the idea behind its proof. Probability is done with integrals throughout; a statistical table, when one appears, is shown to be a precomputed integral.

Boxes: **Intuition** gives the informal reason a result is true; **Connection** links an idea to proper orthogonal decomposition (POD), dynamic mode decomposition (DMD) or computational fluid dynamics (CFD); **Pitfall** marks a place where practice goes wrong. Worked numbers come from scripts in this folder, so they can be re-run.

Contents

1 The surrogate problem and the toolkit **1**

1.1 What a surrogate is . . . 1
1.2 Why the problem is hard: dimension and smoothness . . . 2
1.3 Vectors, inner products and projection . . . 2
1.4 The matrices that matter . . . 4
1.5 Least squares . . . 5
1.6 Numerical facts that decide what can be trusted . . . 6
1.7 Regularisation: trading a little bias for stability . . . 7
1.8 Probability with integrals . . . 8
1.9 The multivariate Gaussian . . . 9
1.10 Estimation and learning . . . 10
1.11 Measuring a surrogate . . . 11
1.12 A map of what follows . . . 12

2 Gaussian processes and kernels **14**

2.1 From finitely many features to a Gaussian process . . . 14
2.2 Kernels: what they encode . . . 15
2.3 Prediction: conditioning on the data . . . 16
2.4 Three names for one object . . . 17
2.5 What the error bar means . . . 18
2.6 Choosing hyper-parameters: the marginal likelihood . . . 18
2.6.1 Where ML-II goes wrong . . . 18
2.7 When a GP is the wrong model . . . 20

Chapter 1

The surrogate problem and the toolkit

This chapter sets up the problem the whole book is about and builds the tools every later chapter uses: geometry of vectors and functions, the few matrix facts that do all the work, least squares, the numerical facts that decide whether a computation can be trusted, probability done with integrals, and the multivariate Gaussian. It ends with Bayesian linear regression, which turns out to be a Gaussian process in disguise; Chapter 2 removes the disguise.

1.1 What a surrogate is

You have a function you can evaluate but not afford to evaluate often: a finite-element run, a CFD solve, a wind-tunnel test. Write it as

$$f : \mathcal{X} \to \mathbb{R}, \qquad \mathcal{X} \subset \mathbb{R}^d, \tag{1.1}$$

where $x \in \mathcal{X}$ collects the d design inputs (geometry parameters, operating conditions) and $f(x)$ is the output you care about. We assume, after rescaling, that $\mathcal{X} = [0,1]^d$, the unit cube; every learner in the study works on this cube.

You run the simulator at n chosen inputs $x_1, \ldots, x_n$ (the *design*) and record

$$y_i = f(x_i) + \varepsilon_i, \qquad i = 1, \ldots, n, \tag{1.2}$$

where ε_i is noise: solver tolerance, mesh effects, stochastic physics, measurement error. For an exact deterministic code $\varepsilon_i = 0$. The data set is $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$.

Definition 1.1 (Surrogate)**.** A surrogate is a function $\hat{f} : \mathcal{X} \to \mathbb{R}$ built from $\mathcal{D}$ that is cheap to evaluate and stands in for f.

A surrogate is used for one of three jobs, and they want different things:

1. **Prediction**: $\hat{f}(x) \approx f(x)$ everywhere in $\mathcal{X}$. Measured by an average error.
2. **Optimisation**: the minimiser of $\hat{f}$ should be a good design for f. Only the region near the optimum matters, and an optimiser that uses gradients also needs $\nabla \hat{f} \approx \nabla f$ there.
3. **Uncertainty quantification**: statements like "$f(x)$ lies in this interval with 95% probability", or averages of f over uncertain inputs. Needs an honest error estimate, not just a good guess.

The paper's central claim is that the most accurate surrogate for job 1 need not be the best for job 2. This book supplies the mathematics to see why.

Intuition. Every surrogate method is an answer to one question: *which functions do you believe f could be?* With n numbers you cannot pin down a function on a continuum; infinitely many functions pass through any data. A method picks one by preferring some functions over others: smooth ones (Gaussian processes, splines), piecewise-constant ones (trees), sums of simple pieces (additive models), ratios of polynomials (rational methods). This preference is called the *inductive bias*. A method wins exactly when its preference matches the truth.

1.2 Why the problem is hard: dimension and smoothness

Grids explode. Ten samples per input is a modest resolution. In d dimensions a grid needs 10^d points: 100 for $d = 2$, 10^5 for $d = 5$, 10^{10} for $d = 10$. At one CPU-hour per finite-element run, $d = 5$ already costs eleven years.

Space becomes empty and corner-heavy. Inscribe a ball of radius $\frac{1}{2}$ in the unit cube. The share of the cube's volume inside the ball is 0.785 for $d = 2$, 0.164 for $d = 5$, 0.0025 for $d = 10$ and 2.5×10^{-8} for $d = 20$. In high dimension almost all the volume is in the corners, far from the centre, so almost every new point is far from every sample.

Distances stop discriminating. Take 1000 random points in the cube and a random query point. The ratio (largest distance − smallest distance)/smallest distance is about 135 for $d = 2$ but only 3.5 for $d = 10$ and 0.39 for $d = 100$. Methods that weight data by distance (nearest neighbours, inverse-distance weighting, kernels with one length-scale) lose their footing, because "near" and "far" become nearly the same.

How fast can error fall? Suppose f is smooth in the sense that its first s derivatives exist and are bounded. Near any point, f is then matched by its Taylor polynomial of degree $s - 1$ with an error proportional to h^s, where h is the distance to the nearest sample. Spread n points evenly in d dimensions and each one owns a cell of volume about $1/n$, so cells have side $h \approx n^{-1/d}$. Putting these together,

$$\text{error} \;\approx\; C\,h^s \;\approx\; C\,n^{-s/d}, \qquad \text{so} \qquad n \;\approx\; (C/\epsilon)^{d/s} \text{ samples for error } \epsilon. \tag{1.3}$$

For error $\epsilon = 0.01$ with $s = 2$ (and $C = 1$) this is 10 samples in one dimension, 100 in two and 10^{10} in ten. Approximation theory shows this rate is also the best any method can guarantee for the *worst* function of that smoothness, so no algorithm escapes it in general (Figure 1.1).

Intuition. Equation (1.3) is the single most important fact in surrogate modelling. It says three things. Smoother functions are easier (s up). Higher dimension is catastrophic (d up). And the only escape is *structure*: if f secretly depends on fewer effective inputs (a ridge $g(w^\mathsf{T}x)$), is a sum of one-dimensional pieces ($\sum_j g_j(x_j)$), is low-rank ($\prod_j g_j(x_j)$), or is very smooth, then the effective d/s is small. Most of the methods in this book are ways of betting on one kind of structure.

1.3 Vectors, inner products and projection

You met these in a proofs course; here each property is restated with the reason it holds, because later chapters lean on the reasons, not the names.

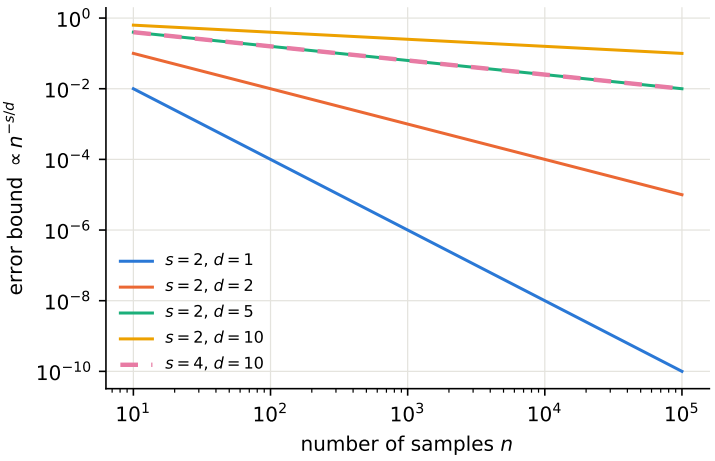


Figure 1.1: The rate $n^{-s/d}$ of equation (1.3). Smoothness s buys speed; dimension d takes it away. Only the ratio s/d matters: doubling the smoothness at $d = 10$ (pink, dashed) gives exactly the curve of halving the dimension to $d = 5$ (green, underneath it).

Vector spaces and bases. A vector space is a set where you can add elements and scale them by numbers, and the usual rules hold. A *basis* $\{v_1, \dots, v_k\}$ is a set of vectors such that every vector is a combination $c_1 v_1 + \cdots + c_k v_k$ in *exactly one* way. "At least one way" means the basis spans the space; "at most one way" means no basis vector is a combination of the others (linear independence). The numbers c_j are the *coordinates*. All bases of a space have the same size, the *dimension*: if one basis were smaller, you could rewrite the larger one in terms of it and find a dependency.

Functions are vectors. Functions on $[0, 1]^d$ add and scale pointwise, so they form a vector space. The polynomials of degree at most p in one variable have basis $1, t, t^2, \dots, t^p$, so dimension $p + 1$. Fitting a surrogate is choosing coordinates in some space of functions. The space of *all* functions has no finite basis, which is why every method restricts itself somehow.

Definition 1.2 (Inner product and norm)**.** An inner product $\langle u, v\rangle$ is a way to multiply two vectors to a number that is symmetric ($\langle u, v\rangle = \langle v, u\rangle$), linear in each argument, and positive ($\langle u, u\rangle > 0$ for $u \neq 0$). It defines a length $\|u\| = \sqrt{\langle u, u\rangle}$ and an angle. In $\mathbb{R}^n$: $\langle u, v\rangle = u^{\mathsf{T}} v = \sum_i u_i v_i$. For functions on $[0, 1]$:

$$\langle g, h\rangle = \int_0^1 g(t)\, h(t)\, dt, \qquad \|g\|^2 = \int_0^1 g(t)^2\, dt, \tag{1.4}$$

so the "length" of a function is its root-mean-square size. A surrogate's mean-squared error is the squared length $\|\hat{f} - f\|^2$ of the error function.

Result 1.3 (Cauchy–Schwarz)**.** $|\langle u, v\rangle| \leq \|u\|\, \|v\|$, with equality only when u and v are parallel.

Intuition. For every number t, the squared length $\|u - tv\|^2 = \|u\|^2 - 2t\langle u, v\rangle + t^2\|v\|^2$ is at least zero. A quadratic in t that is never negative cannot have two real roots, so its discriminant $4\langle u, v\rangle^2 - 4\|u\|^2\|v\|^2$ is not positive. That is the inequality. It is what lets us define the angle $\cos\theta = \langle u, v\rangle / (\|u\|\|v\|)$: the ratio is guaranteed to lie in $[-1, 1]$.

Orthogonality and Gram–Schmidt. Vectors are orthogonal when $\langle u, v\rangle = 0$. An orthonormal basis $\{e_j\}$ (orthogonal, unit length) makes coordinates trivial: the coordinate of u along e_j is just $\langle u, e_j\rangle$, because taking the inner product of $u = \sum_k c_k e_k$ with e_j kills every term except c_j. Gram–Schmidt turns any basis $v_1, v_2, \ldots$ into an orthonormal one:

$$u_k = v_k - \sum_{j<k} \langle v_k, e_j\rangle\, e_j, \qquad e_k = u_k/\|u_k\|. \tag{1.5}$$

Why it works: $\langle v_k, e_j\rangle\, e_j$ is the part of v_k that points along e_j; subtracting all those parts leaves only the genuinely new direction. Applied to $1, t, t^2, \ldots$ on $[-1, 1]$ with the integral inner product, Gram–Schmidt produces (up to scaling) the *Legendre polynomials* $P_0 = 1$, $P_1 = t$, $P_2 = \frac{1}{2}(3t^2 - 1), \ldots$ — an orthogonal basis of polynomials. Section 1.6 shows why that matters numerically.

Result 1.4 (Orthogonal projection)**.** Let S be a subspace and y any vector. There is exactly one $p \in S$ closest to y, and it is characterised by the residual being orthogonal to the whole subspace: $\langle y - p, s\rangle = 0$ for every $s \in S$. If $\{e_j\}$ is an orthonormal basis of S, then $p = \sum_j \langle y, e_j\rangle\, e_j$.

Intuition. Pythagoras. Suppose $y - p$ is orthogonal to S, and take any other $s \in S$. Then $p - s$ lies in S, so it is orthogonal to $y - p$, and

$$\|y - s\|^2 = \|(y - p) + (p - s)\|^2 = \|y - p\|^2 + \|p - s\|^2 \;\geq\; \|y - p\|^2.$$

Any other candidate is farther away by exactly the extra leg $\|p - s\|$. The closest point is the foot of the perpendicular, just as in the plane.

Connection. POD is projection. You collect snapshots of a flow field as columns of a matrix, find an orthonormal basis (the POD modes) of a low-dimensional subspace, and represent each snapshot by its projection $\sum_j \langle u, e_j\rangle\, e_j$. The POD modes are chosen so that, averaged over the snapshots, the projection error is as small as possible for that number of modes (Result 1.8 below). Least-squares fitting, Galerkin methods and Gaussian-process prediction are all projections in suitably chosen spaces.

1.4 The matrices that matter

Symmetric matrices. A matrix is symmetric when $A = A^{\mathsf{T}}$. Covariance matrices, Gram matrices $A^{\mathsf{T}}A$ and kernel matrices are all symmetric, so this case dominates the book.

Result 1.5 (Spectral theorem)**.** A real symmetric $n \times n$ matrix has n real eigenvalues and an orthonormal basis of eigenvectors: $A = Q\Lambda Q^{\mathsf{T}}$, where Q has the eigenvectors as columns ($Q^{\mathsf{T}}Q = I$) and Λ is diagonal.

Intuition. Two eigenvectors with different eigenvalues are automatically orthogonal: if $Av_1 = \lambda_1 v_1$ and $Av_2 = \lambda_2 v_2$, then $\lambda_1\langle v_1, v_2\rangle = \langle Av_1, v_2\rangle = \langle v_1, Av_2\rangle = \lambda_2\langle v_1, v_2\rangle$, using symmetry in the middle step. With $\lambda_1 \neq \lambda_2$ that forces $\langle v_1, v_2\rangle = 0$. Geometrically, a symmetric matrix does nothing but stretch space along perpendicular axes: in the coordinates of its eigenvectors it is a diagonal scaling.

Definition 1.6 (Positive (semi)definite)**.** A symmetric A is positive definite (PD) if $x^{\mathsf{T}}Ax > 0$ for every $x \neq 0$, and positive semidefinite (PSD) if $x^{\mathsf{T}}Ax \geq 0$. Write $x = Qc$ in eigen-coordinates; then $x^{\mathsf{T}}Ax = \sum_i \lambda_i c_i^2$, so *PD* $\iff$ *all eigenvalues positive* and *PSD* $\iff$ *all eigenvalues nonnegative.*

The quantity $x^{\mathsf{T}}Ax$ is like an energy: PD means every direction costs energy. Two facts carry most of the weight later:

- **Gram matrices are PSD.** For any matrix Φ, $x^\mathsf{T}(\Phi^\mathsf{T}\Phi)x = \|\Phi x\|^2 \geq 0$. They are PD exactly when Φ has independent columns (otherwise some $x \neq 0$ has $\Phi x = 0$).
- **Covariance matrices are PSD.** $\mathrm{Var}(a^\mathsf{T}X) = a^\mathsf{T}\,\mathrm{Cov}(X)\,a \geq 0$ because a variance cannot be negative (Section 1.8).

Result 1.7 (Singular value decomposition)**.** Every $m \times n$ matrix factors as $A = U\Sigma V^\mathsf{T}$ with U $(m \times m)$ and V $(n \times n)$ orthogonal and Σ diagonal with entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$, the singular values. Equivalently $Av_j = \sigma_j u_j$: A sends the orthonormal directions v_j to the orthonormal directions u_j, stretched by σ_j.

Intuition. Any matrix is rotate, stretch, rotate. The right singular vectors v_j are the eigenvectors of the symmetric PSD matrix $A^\mathsf{T}A$ (so they exist by the spectral theorem), and σ_j^2 are its eigenvalues; then $u_j = Av_j/\sigma_j$ come out orthonormal automatically. The number of nonzero σ_j is the rank: how many independent directions A really uses.

Result 1.8 (Best low-rank approximation, Eckart–Young)**.** Keeping the first k terms, $A_k = \sum_{j \leq k} \sigma_j u_j v_j^\mathsf{T}$, gives the closest rank-k matrix to A, and the squared error is $\sum_{j>k} \sigma_j^2$.

Connection. This is POD. With snapshots as columns of A, the left singular vectors u_j are the POD modes, σ_j^2 is the "energy" each mode captures, and truncating at k modes is optimal in the least-squares sense with the discarded energy as the error. DMD starts from exactly this truncated SVD of the snapshot matrix before fitting its linear operator. The low-rank surrogates of later chapters (functional SVD, tensor trains) are the same idea applied to a function of several variables instead of a snapshot matrix.

1.5 Least squares

Most surrogates contain a least-squares problem. Choose basis functions $\phi_1, \ldots, \phi_k$ (monomials, Legendre polynomials, radial functions…) and look for $\hat{f}(x) = \sum_j w_j \phi_j(x)$. Collect $\Phi_{ij} = \phi_j(x_i)$ in the $n \times k$ *design matrix* and solve

$$\min_{w \in \mathbb{R}^k} \|y - \Phi w\|^2 = \min_{w} \sum_{i=1}^{n} \Big(y_i - \sum_j w_j \phi_j(x_i) \Big)^2. \tag{1.6}$$

The vectors Φw form a subspace of $\mathbb{R}^n$ (the column space of Φ), so this is projecting y onto it. By Result 1.4 the residual must be orthogonal to every column:

$$\Phi^\mathsf{T}(y - \Phi w) = 0 \quad \Longleftrightarrow \quad \Phi^\mathsf{T}\Phi\, w = \Phi^\mathsf{T} y. \tag{1.7}$$

These are the *normal equations*. They have a unique solution exactly when the columns are independent (the Gram matrix $\Phi^\mathsf{T}\Phi$ is then PD, hence invertible).

With the SVD $\Phi = U\Sigma V^\mathsf{T}$ (thin form, $\sigma_j > 0$),

$$w = V\Sigma^{-1}U^\mathsf{T} y = \sum_j \frac{u_j^\mathsf{T} y}{\sigma_j}\, v_j. \tag{1.8}$$

Read it as: express y in the output directions u_j, divide by how much Φ stretches each direction, and map back. The danger is visible already: a tiny σ_j divides, so any noise in y along u_j is blown up.

Connection. DMD's best-fit linear operator, $X' \approx AX$, is a least-squares problem of exactly this type, and "exact DMD" solves it through the truncated SVD of X, discarding small σ_j for exactly the reason above.

1.6 Numerical facts that decide what can be trusted

Floating point. A computer stores a real number x as $\mathrm{fl}(x) = x(1+\delta)$ with $|\delta| \le u$, the *unit round-off*; for standard double precision $u = 2^{-53} \approx 1.1 \times 10^{-16}$. Every arithmetic operation adds such a relative error. This is harmless until two nearly equal numbers are subtracted: the leading digits cancel and only the rounding errors are left (*cancellation*).

Conditioning. A problem is *ill-conditioned* if small changes in its data cause large changes in its answer, whatever algorithm you use. For a square linear system $Aw = b$ the amplification factor is the *condition number*

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}}, \qquad \frac{\|\delta w\|}{\|w\|} \lesssim \kappa(A)\, \frac{\|\delta b\|}{\|b\|}. \tag{1.9}$$

Why: by (1.8) a perturbation of b along the weakest direction is divided by $\sigma_{\min}$, while b itself can lie along the strongest one. Rule of thumb: you lose about $\log_{10} \kappa$ of your 16 digits.

Example 1.9 (Bases matter)**.** Fit polynomials of degree p on $[-1, 1]$ from 100 equally spaced points. Condition numbers of the design matrix:

degree p	monomials $1, t, \ldots, t^p$	Legendre $P_0, \ldots, P_p$	normal equations, monomials
5	4.2×10^1	3.2	1.8×10^3
10	2.9×10^3	4.4	8.6×10^6
15	2.2×10^5	5.5	4.8×10^{10}
20	1.7×10^7	6.4	3.0×10^{14}

The function space is identical; only the basis differs. Monomials t^{15} and t^{17} look almost the same on $[-1, 1]$ (nearly parallel vectors), so separating them is ill-conditioned; Legendre polynomials are orthogonal, so nothing needs separating. The last column shows the second lesson: forming $\Phi^{\mathsf{T}}\Phi$ squares the condition number ($\kappa(\Phi^{\mathsf{T}}\Phi) = \kappa(\Phi)^2$, since its singular values are σ_j^2). At degree 20 that leaves two correct digits. Solve least squares through QR or the SVD, never by forming the normal equations.

Stability. Conditioning belongs to the problem; *stability* belongs to the algorithm. A stable algorithm gives an answer as accurate as the conditioning allows. Forming $\Phi^{\mathsf{T}}\Phi$ is an unstable route to a least-squares solution (it squares κ before solving); QR is a stable one.

Cholesky, and why we never invert. A PD matrix factors uniquely as $A = LL^{\mathsf{T}}$ with L lower triangular (*Cholesky*). It exists because positivity guarantees every pivot is positive, so the square roots in the construction are real. It costs $n^3/3$ operations, half of a general factorisation. To solve $Aw = b$, solve $Lz = b$ then $L^{\mathsf{T}}w = z$, each by substitution in n^2 operations. Two by-products appear constantly in Gaussian-process work: $\log\det A = 2\sum_i \log L_{ii}$ (stable, never overflows), and a cheap test of whether A is numerically PD (the factorisation fails if not). Computing A^{-1} explicitly costs more and is less accurate than solving; "$A^{-1}b$" in a formula always means "solve".

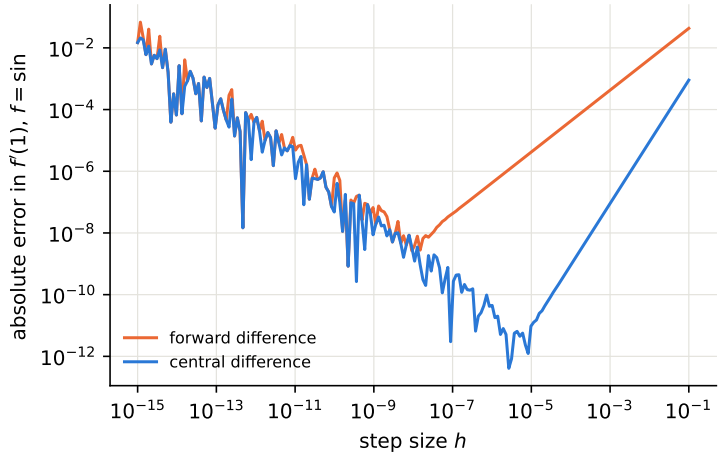


Figure 1.2: Finite-difference error versus step size for $f'(1)$, $f = \sin$. Right of each minimum, truncation error dominates (slope 1 forward, slope 2 central); left of it, rounding dominates (slope -1). The study's gradient metric uses central differences with $h = 10^{-5}$, near the bottom of the blue V.

Finite differences: the trade-off you asked about. To approximate $f'(x)$ by $\big(f(x+h) - f(x)\big)/h$ there are two errors. Taylor's theorem gives a truncation error of about $\frac{h}{2}|f''|$, which shrinks with h. Rounding gives about $2u|f|/h$, which grows as h shrinks, because $f(x+h)-f(x)$ is a cancellation. The total is smallest when the two balance, at $h \approx \sqrt{u} \approx 10^{-8}$, leaving about 10^{-8} error. The central difference $\big(f(x+h) - f(x-h)\big)/(2h)$ has truncation error $\frac{h^2}{6}|f'''|$; balancing against $u|f|/h$ gives $h \approx u^{1/3} \approx 5 \times 10^{-6}$ and an error near 10^{-11}. Figure 1.2 shows both for $f = \sin$ at $x = 1$: the V shapes are exactly this trade-off.

Pitfall. The finite-difference error that matters in practice is not this one. An optimizer differencing a surrogate with a practitioner's step (10^{-3}, say) across a kink, a jump or a noisy surface gets an error unrelated to round-off. That is why the study measures gradient accuracy with a tiny, accurate step, and treats the optimizer's own gradient source as a separate factor in the optimisation experiment.

1.7 Regularisation: trading a little bias for stability

When Φ has small singular values, (1.8) amplifies noise. *Ridge regression* (Tikhonov regularisation) adds a penalty:

$$\min_w \|y - \Phi w\|^2 + \lambda \|w\|^2 \quad \Longrightarrow \quad (\Phi^\mathsf{T}\Phi + \lambda I)\, w = \Phi^\mathsf{T} y \quad \Longrightarrow \quad w = \sum_j \frac{\sigma_j}{\sigma_j^2 + \lambda}\, (u_j^\mathsf{T} y)\, v_j. \tag{1.10}$$

Compare with (1.8): the factor $1/\sigma_j$ has become $\sigma_j/(\sigma_j^2 + \lambda)$. For strong directions ($\sigma_j^2 \gg \lambda$) nothing changes; for weak ones ($\sigma_j^2 \ll \lambda$) the factor is about σ_j/λ, which goes to zero instead of blowing up. Ridge is a smooth version of throwing away small singular values, the "filter factors" $\sigma_j^2/(\sigma_j^2 + \lambda)$ that you may know from POD-based reduced-order models. The price is *bias*: even with perfect data, w is pulled towards zero. Choosing λ is choosing how much bias to accept for how much stability; Section 1.10 makes this exact.

Every regularised method later in the book — the GP nugget, the Tikhonov denominator of rational fits, the ridge inside alternating least squares — is this same filter in a different basis.

1.8 Probability with integrals

Statistics courses often switch to tables early. Here everything is an integral, and the tables are explained at the end as what they are.

Definition 1.10 (Density)**.** A continuous random variable X is described by a density $p(x) \geq 0$ with $\int p(x)\,dx = 1$. Probabilities are areas under it:

$$\Pr(a \leq X \leq b) = \int_a^b p(x)\,dx. \tag{1.11}$$

The density is not a probability; it is probability per unit length. It can exceed 1; only its integral is bounded.

Expectation, mean, variance. The expected value of any function g of X is a weighted average:

$$\mathbb{E}[g(X)] = \int g(x)\,p(x)\,dx. \tag{1.12}$$

The mean is $\mu = \mathbb{E}[X]$, the variance $\operatorname{Var}(X) = \mathbb{E}[(X-\mu)^2]$, and the standard deviation $\sqrt{\operatorname{Var} X}$. Expanding the square and using linearity of the integral, $\operatorname{Var}(X) = \mathbb{E}[X^2] - \mu^2$. Linearity is the property used most: $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$ always, because an integral of a sum is the sum of integrals.

Several variables. Two variables have a joint density $p(x, y)$, with $\Pr\big((X,Y) \in R\big) = \iint_R p(x,y)\,dx\,dy$. Three operations do everything:

- **Marginalise** (forget y): $p(x) = \int p(x,y)\,dy$. Integrate out what you do not care about.
- **Condition** (learn that $X = x$): $p(y \mid x) = p(x,y)/p(x)$. Take the slice of the joint density at $X = x$ and rescale it so it integrates to one; the divisor is exactly the marginal.
- **Bayes' rule**: writing the joint both ways, $p(x,y) = p(y \mid x)\,p(x) = p(x \mid y)\,p(y)$, gives

$$p(\theta \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \theta)\;p(\theta)}{p(\mathcal{D})}, \qquad p(\mathcal{D}) = \int p(\mathcal{D} \mid \theta)\,p(\theta)\,d\theta. \tag{1.13}$$

In words: what you believe about unknowns θ after seeing data (the *posterior*) is what you believed before (the *prior*) times how well each θ explains the data (the *likelihood*), rescaled to integrate to one. Nothing more is going on in any Bayesian method in this book.

Independence and covariance. X and Y are independent when $p(x,y) = p(x)\,p(y)$: learning one tells you nothing about the other. Their covariance is $\operatorname{Cov}(X,Y) = \mathbb{E}[(X-\mu_X)(Y-\mu_Y)]$, which measures linear co-movement. For a vector X, the covariance matrix has entries $\operatorname{Cov}(X_i, X_j)$, and for any fixed vector a, $\operatorname{Var}(a^{\mathsf{T}}X) = a^{\mathsf{T}}\operatorname{Cov}(X)\,a$ — which is why covariance matrices are PSD.

Connection (What a statistical table is)**.** A "z-table" lists $\Phi(z) = \int_{-\infty}^{z} \frac{1}{\sqrt{2\pi}} e^{-t^2/2}\,dt$ for many z: the area under the standard bell curve to the left of z. This integral has no formula in elementary functions, so it was computed numerically once and printed. "Look up 1.96 in the table" means "the area to the left of 1.96 is 0.975", so the central area between -1.96 and 1.96 is 0.95: that is where 95% intervals come from. t-tables and χ^2-tables are the same thing for other densities. Today one calls `scipy.stats.norm.cdf`.

1.9 The multivariate Gaussian

Definition 1.11 (Gaussian distribution)**.** $X \in \mathbb{R}^d$ is Gaussian with mean μ and covariance Σ (PD), written $X \sim \mathcal{N}(\mu, \Sigma)$, if

$$p(x) = \frac{1}{(2\pi)^{d/2}\,\det(\Sigma)^{1/2}} \exp\Big(-\tfrac{1}{2}(x-\mu)^{\mathsf{T}}\Sigma^{-1}(x-\mu)\Big). \tag{1.14}$$

Reading the formula. The exponent contains $(x-\mu)^{\mathsf{T}}\Sigma^{-1}(x-\mu)$, a squared distance from the mean measured in units of the spread: the *Mahalanobis distance*. In the eigenvector coordinates of Σ (spectral theorem) it becomes $\sum_i c_i^2/\lambda_i$: each direction is measured relative to its own variance λ_i, so the level sets are ellipsoids with axes along the eigenvectors and half-lengths $\sqrt{\lambda_i}$. In those same coordinates the density is a product of d one-dimensional bell curves, each integrating to $\sqrt{2\pi\lambda_i}$; the product is $(2\pi)^{d/2}\prod_i \sqrt{\lambda_i} = (2\pi)^{d/2}\det(\Sigma)^{1/2}$, which is the normalising constant. Nothing in (1.14) is arbitrary.

Four properties make Gaussians the workhorse of uncertainty quantification.

Result 1.12 (Linear maps preserve Gaussianity)**.** If $X \sim \mathcal{N}(\mu, \Sigma)$ then $AX + b \sim \mathcal{N}(A\mu + b,\ A\Sigma A^{\mathsf{T}})$.

Why: the mean and covariance follow from linearity of expectation; Gaussianity survives because substituting a linear change of variables into a quadratic exponent leaves a quadratic exponent. In particular a sum of jointly Gaussian variables is Gaussian, so a prediction that is a weighted sum of Gaussian unknowns is Gaussian too.

Result 1.13 (Marginals)**.** Split $X = (a, b)$ with $\mu = (\mu_a, \mu_b)$ and $\Sigma = \begin{pmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{pmatrix}$. Then $a \sim \mathcal{N}(\mu_a, \Sigma_{aa})$: to forget b, delete its rows and columns. (This is the linear map that picks out a.)

Result 1.14 (Uncorrelated means independent, for Gaussians)**.** If $\Sigma_{ab} = 0$, then a and b are independent.

Why: with a block-diagonal Σ, the inverse is block diagonal and the exponent splits as a sum of an a-part and a b-part, so the density factorises. (For non-Gaussian variables, zero covariance does *not* imply independence.)

Result 1.15 (Conditioning — the engine of Gaussian-process prediction)**.** For (a, b) jointly Gaussian as above,

$$a \mid b \ \sim\ \mathcal{N}\Big(\mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(b-\mu_b),\ \ \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}\Big). \tag{1.15}$$

Intuition. Remove from a the part that b can explain linearly. Define $z = a - \Sigma_{ab}\Sigma_{bb}^{-1}b$. Its covariance with b is $\Sigma_{ab} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{bb} = 0$, and (z, b) is jointly Gaussian (a linear map of (a, b)), so by the previous

result z is *independent* of b. Now write $a = z + \Sigma_{ab}\Sigma_{bb}^{-1}b$. Once b is known, the second term is a known constant and z keeps its own distribution, because it does not care about b. So $a \mid b$ has mean $\mathbb{E}[z] + \Sigma_{ab}\Sigma_{bb}^{-1}b = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(b - \mu_b)$ and covariance $\mathrm{Cov}(z) = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$ (expand $\mathrm{Cov}(z)$ to check). The mean shifts by a regression on the surprise $b - \mu_b$; the covariance shrinks by the part b explains, and it does not depend on the value of b at all.

Connection. Chapter 2 applies (1.15) with b = the observed outputs and a = the unknown output at a new point. The "GP posterior mean and variance" are nothing but this formula. Numerically, Σ_{bb}^{-1} is never formed: one Cholesky factorisation of Σ_{bb} serves both the mean and the variance.

1.10 Estimation and learning

Estimators, bias and variance. An estimator $\hat{\theta}$ is a recipe that turns data into a guess of an unknown θ. Because the data are random (noise, random design), $\hat{\theta}$ is a random variable. Its mean-squared error splits exactly into two parts:

$$\mathbb{E}\big[(\hat{\theta} - \theta)^2\big] = \underbrace{\big(\mathbb{E}[\hat{\theta}] - \theta\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}\big[(\hat{\theta} - \mathbb{E}\hat{\theta})^2\big]}_{\text{variance}}. \tag{1.16}$$

Why: add and subtract $\mathbb{E}[\hat{\theta}]$ inside the square; the cross term has expectation zero because $\mathbb{E}[\hat{\theta} - \mathbb{E}\hat{\theta}] = 0$. Bias is being wrong on average; variance is being unstable from one data set to the next. Ridge regression accepts bias to cut variance; a very flexible model has low bias and high variance. A surrogate's error behaves the same way at every point x.

Maximum likelihood is least squares. Assume the noise in (1.2) is independent Gaussian with variance σ^2 and $f(x) = \phi(x)^{\mathsf{T}}w$. The likelihood of the data is the product of the individual densities,

$$p(y \mid w) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\Big(-\frac{(y_i - \phi(x_i)^{\mathsf{T}}w)^2}{2\sigma^2}\Big), \qquad -\log p(y \mid w) = \frac{1}{2\sigma^2}\|y - \Phi w\|^2 + \text{const.} \tag{1.17}$$

Maximising the likelihood is therefore exactly least squares. Least squares is the right fit for Gaussian noise; for heavy-tailed noise (occasional outliers) other losses fit better, which is part of why some methods do better under the study's heavy-tailed noise condition.

Maximum a posteriori is ridge. Add a prior belief that weights are modest: $w \sim \mathcal{N}(0, \tau^2 I)$. By Bayes' rule (1.13), the log posterior is the log likelihood plus the log prior:

$$-\log p(w \mid y) = \frac{1}{2\sigma^2}\|y - \Phi w\|^2 + \frac{1}{2\tau^2}\|w\|^2 + \text{const.} \tag{1.18}$$

Its minimiser is ridge regression (1.10) with $\lambda = \sigma^2/\tau^2$: noise level over prior size. Regularisation is not a trick; it is a prior belief written as a penalty.

Bayesian linear regression: the whole posterior. Instead of the single best w, keep the full posterior. Treat w and $y = \Phi w + \varepsilon$ as jointly Gaussian. By the linear-map rule, $y \sim \mathcal{N}(0,\ \tau^2\Phi\Phi^{\mathsf{T}} + \sigma^2 I)$ and $\mathrm{Cov}(w, y) = \tau^2\Phi^{\mathsf{T}}$. Conditioning (1.15) on the observed y gives

$$w \mid y \sim \mathcal{N}(m, S), \qquad m = \tau^2\Phi^{\mathsf{T}}\big(\tau^2\Phi\Phi^{\mathsf{T}} + \sigma^2 I\big)^{-1}y, \qquad S = \tau^2 I - \tau^2\Phi^{\mathsf{T}}\big(\tau^2\Phi\Phi^{\mathsf{T}} + \sigma^2 I\big)^{-1}\Phi\,\tau^2. \tag{1.19}$$

A prediction at a new point, $\hat{f}(x_*) = \phi(x_*)^\mathsf{T} w$, is a linear map of a Gaussian, hence Gaussian:

$$\hat{f}(x_*) \mid y \sim \mathcal{N}\Big(\underbrace{\phi_*^\mathsf{T} m}_{\text{prediction}} \;,\; \underbrace{\phi_*^\mathsf{T} S\, \phi_*}_{\text{its uncertainty}} \Big), \qquad \phi_* = \phi(x_*). \tag{1.20}$$

Expand $\phi_*^\mathsf{T} m = \tau^2 \phi_*^\mathsf{T} \Phi^\mathsf{T} (\tau^2 \Phi\Phi^\mathsf{T} + \sigma^2 I)^{-1} y$. The data enter *only* through inner products $\tau^2 \phi(x)^\mathsf{T}\phi(x')$ between feature vectors: $\Phi\Phi^\mathsf{T}$ is the matrix of those inner products between design points, and $\Phi\phi_*$ the inner products with the new point. Call this $k(x, x') = \tau^2 \phi(x)^\mathsf{T}\phi(x')$. Then

$$\text{mean}(x_*) = k_*^\mathsf{T} (K + \sigma^2 I)^{-1} y, \qquad \text{variance}(x_*) = k(x_*, x_*) - k_*^\mathsf{T} (K + \sigma^2 I)^{-1} k_*, \tag{1.21}$$

with $K_{ij} = k(x_i, x_j)$ and $(k_*)_i = k(x_i, x_*)$. This is a Gaussian process with kernel k. Chapter 2 starts here and asks: what if we choose k directly, so that the feature space is infinite-dimensional and we never write ϕ down?

1.11 Measuring a surrogate

The quantity we want is the true average error over the domain,

$$\text{MSE}(\hat{f}) = \int_{\mathcal{X}} \big(\hat{f}(x) - f(x)\big)^2 \, dx = \|\hat{f} - f\|^2. \tag{1.22}$$

It is an integral, so we estimate it the way any integral is estimated: average the integrand at many points. The study uses 4096 quasi-random test points and reports

$$\text{NRMSE} = \frac{\sqrt{\frac{1}{N}\sum_{t=1}^{N} \big(\hat{f}(x_t) - f(x_t)\big)^2}}{\text{sd}\,\big(f(x_1), \ldots, f(x_N)\big)}, \tag{1.23}$$

scaled so that 1 means "no better than predicting the mean" and 0 is perfect.

Result 1.16 (Noisy tests have a floor)**.** If the test outputs are themselves noisy, $\tilde{y}_t = f(x_t) + \varepsilon_t$ with noise of variance σ^2 independent of the surrogate, then

$$\mathbb{E}\big[(\hat{f}(x) - \tilde{y})^2\big] = \mathbb{E}\big[(\hat{f}(x) - f(x))^2\big] + \sigma^2. \tag{1.24}$$

Why: write $\hat{f} - \tilde{y} = (\hat{f} - f) - \varepsilon$, expand the square, and the cross term vanishes because ε has mean zero and is independent of $\hat{f} - f$. Error measured against noisy data can never fall below the noise variance, and when every method is near that floor their differences are compressed. This is exactly the thesis-review finding (bug B-01): scoring against noisy cross-validation folds made the GP's lead over boosted trees look 1.2× when on clean truth it was 2.4×. With a simulator we can evaluate f itself, so the study scores against clean truth.

Cross-validation. Without clean test data, hold out part of the data, fit on the rest and score on the held-out part, rotating which part is held out. It estimates the error on new data from the same distribution, noise floor included. Choosing hyper-parameters (a ridge λ, a polynomial degree) by such a hold-out is used throughout the study's learners.

1.12 A map of what follows

Every later chapter is one answer to "which functions do you believe in?" (the inductive bias), with the mathematics that follows from it.

Family	Functions it prefers	Structure it exploits (Section 1.2)
Gaussian processes, kernels, RBF (Ch. 2)	smooth, with a smoothness set by the kernel	smoothness s
Polynomial chaos, sparse regression	low-degree polynomials, few active terms	smoothness, sparsity
Splines, GAMs, MARS	piecewise polynomials; sums of 1-D pieces	additivity
Trees, forests, boosting	piecewise constant on boxes	axis-aligned jumps
Neural networks, KANs	compositions of simple functions	compositional structure
Ridge / active subspaces	$g(W^\mathsf{T}x)$ with few directions	low effective dimension
Low-rank, tensor trains	products of 1-D functions	separability
Rational (AAA, p/q)	ratios of polynomials	poles, knees, resonances
Bayesian NNs, BART, PFNs	a prior over functions, learned or explicit	whatever the prior encodes

Notes and questions for later

1. **Why $\mathcal{X} \subset \mathbb{R}^d$?** For a design study the inputs are the chosen parameters, so they are coordinates by construction. When the meaningful inputs lie on a lower-dimensional set (constrained designs, shapes, data on a manifold), nothing requires knowing that set: methods only use points and distances in the ambient coordinates. Embedding theorems say a representation always exists: Whitney, a smooth m-manifold embeds smoothly in $\mathbb{R}^{2m}$; Menger–Nöbeling, a compact space of topological dimension m embeds continuously in $\mathbb{R}^{2m+1}$. Continuity is enough to embed; smoothness matters only for the *rate* (the s in $n^{-s/d}$), which drops near corners and cusps. Kernel and nearest-neighbour methods adapt to the intrinsic dimension m (the "manifold hypothesis"). Takens' delay embedding is the same idea behind delay-coordinate DMD. A later chapter on geometry and dimension treats this properly.

2. **Why $f : \mathcal{X} \to \mathbb{R}$ and not $\mathbb{R}^n$?** A design objective is scalar. Vector and field outputs (independent models, multi-output GPs and co-kriging, POD coefficients plus regression, i.e. non-intrusive ROMs) get their own chapter.

3. **Eckart–Young says how to truncate at a given k, not how to choose k.** Choosing k from noisy data is Gavish and Donoho's optimal hard threshold ($\tau = (4/\sqrt{3})\sqrt{n}\,\sigma$ for a square matrix with known noise; about 2.858× the median singular value when σ is unknown). It belongs with robust Padé and the SVD rank stopping of AAA in the low-rank and rational chapter, together with the nuclear norm (sum of singular values), the convex "L1 for rank".

Summary of the formulas to keep

- Rate: error $\approx n^{-s/d}$; structure lowers the effective d/s.

- Projection: the best approximation from a subspace leaves a residual orthogonal to it.
- Least squares: $\Phi^\mathsf{T}\Phi w = \Phi^\mathsf{T} y$, but solve by QR/SVD; $w = \sum_j (u_j^\mathsf{T} y/\sigma_j) v_j$.
- Conditioning: $\kappa = \sigma_{\max}/\sigma_{\min}$; normal equations square it; orthogonal bases fix it.
- Ridge: filter factors $\sigma_j^2/(\sigma_j^2 + \lambda)$; equals MAP with prior variance σ^2/λ.
- Bayes: posterior $\propto$ likelihood $\times$ prior; conditioning is slicing and renormalising.
- Gaussian conditioning: mean $\mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(b - \mu_b)$, covariance $\Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$.
- Bias^2 + variance = MSE; noisy test error = true error + noise variance.

Chapter 2

Gaussian processes, kernels and radial basis functions

Chapter 1 ended with Bayesian linear regression and noticed that its predictions depend on the features only through inner products $k(x, x') = \tau^2 \phi(x)^\mathsf{T} \phi(x')$. This chapter takes that seriously: choose k directly, let the feature space be infinite-dimensional, and the result is a Gaussian process (GP). We then show that kriging, kernel ridge regression and radial-basis-function interpolation are the same object seen from three disciplines, derive what a GP's error bars really mean, and explain the failure modes the study measured.

2.1 From finitely many features to a Gaussian process

Definition 2.1 (Gaussian process)**.** A Gaussian process is a random function f such that, for any finite set of points $x_1, \dots, x_n$, the vector $\big(f(x_1), \dots, f(x_n)\big)$ is jointly Gaussian. It is fixed by a mean function $m(x) = \mathbb{E}[f(x)]$ and a covariance function (*kernel*) $k(x, x') = \mathrm{Cov}\big(f(x), f(x')\big)$. We write $f \sim \mathcal{GP}(m, k)$.

Think of a GP as an infinitely long Gaussian vector indexed by x instead of by an integer. You never handle the infinite object: every computation touches only the finitely many points you have, and the marginal rule of Chapter 1 ("delete the rows and columns you do not need") guarantees that answers do not depend on points you did not ask about.

Connection (Mercer and Karhunen–Loève: a GP is POD of a random field)**.** A valid kernel can be expanded as $k(x, x') = \sum_j \lambda_j \, \psi_j(x) \, \psi_j(x')$, where ψ_j are the eigenfunctions of the integral operator $(\mathcal{K}g)(x) = \int k(x, x') g(x') \, dx'$ and $\lambda_j \geq 0$ its eigenvalues (*Mercer's theorem*, the continuous version of the spectral theorem for the symmetric matrix K). Then a GP sample can be written

$$f(x) = \sum_j \sqrt{\lambda_j} \, \xi_j \, \psi_j(x), \qquad \xi_j \sim \mathcal{N}(0, 1) \text{ independent} \tag{2.1}$$

(the *Karhunen–Loève expansion*). This is exactly POD applied to a random field instead of a snapshot matrix: ψ_j are the modes, λ_j the energies, and the decay of λ_j says how many modes matter. It is also Bayesian linear regression with features $\sqrt{\lambda_j}\psi_j$ and infinitely many of them. A GP is the limit of Chapter 1's model in which you stop truncating.

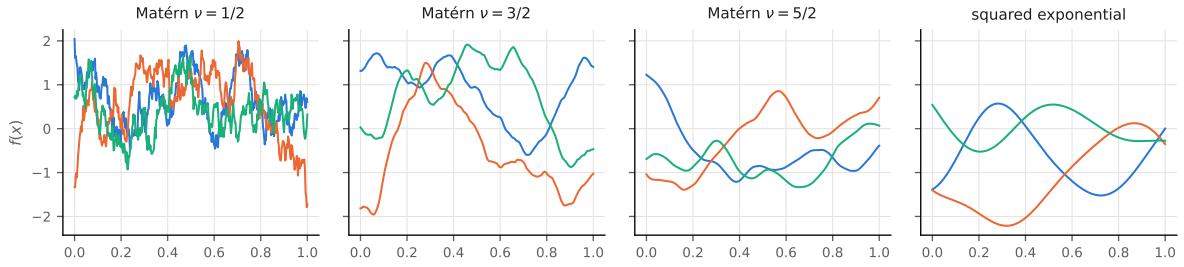


Figure 2.1: Three random functions from each of four GP priors, all with length-scale $\ell = 0.2$. Same correlation distance, very different smoothness.

2.2 Kernels: what they encode

What makes a kernel valid. Any covariance matrix must be PSD, so a function k can be a kernel only if every matrix $K_{ij} = k(x_i, x_j)$ is PSD, for every choice of points. Sums, products and positive multiples of valid kernels are valid (a sum of covariances is the covariance of a sum of independent processes; a product is the covariance of a product of independent processes), which is how kernels are built.

Stationary kernels. Most surrogate work uses kernels that depend only on the separation $r = \|x - x'\|/\ell$, with a *length-scale* ℓ and a signal variance σ_f^2:

$$\text{squared exponential (SE):} \quad k(r) = \sigma_f^2 \exp(-r^2/2), \tag{2.2}$$

$$\text{Matérn } \nu = \tfrac{1}{2}: \quad k(r) = \sigma_f^2\, e^{-r}, \tag{2.3}$$

$$\text{Matérn } \nu = \tfrac{3}{2}: \quad k(r) = \sigma_f^2 (1 + \sqrt{3}\, r)\, e^{-\sqrt{3} r}, \tag{2.4}$$

$$\text{Matérn } \nu = \tfrac{5}{2}: \quad k(r) = \sigma_f^2 \big(1 + \sqrt{5}\, r + \tfrac{5}{3} r^2\big) e^{-\sqrt{5} r}. \tag{2.5}$$

The length-scale is the distance over which f "forgets" its value: correlation has fallen to about $e^{-1/2}$ (SE) when $r = 1$. The *automatic relevance determination* (ARD) version uses one length-scale per input, $r^2 = \sum_j (x_j - x'_j)^2/\ell_j^2$; a large ℓ_j means f barely changes along input j.

Smoothness is the choice that matters. Figure 2.1 shows draws from four GPs with the same length-scale. Matérn $\nu = \frac{1}{2}$ gives continuous but nowhere-differentiable paths (it is the velocity of a particle in Brownian motion, roughly); $\nu = \frac{3}{2}$ gives once-differentiable paths; $\nu = \frac{5}{2}$ twice; SE ($\nu \to \infty$) infinitely differentiable. In general Matérn-ν paths have $\lceil \nu \rceil - 1$ derivatives. So ν is the s of Chapter 1's rate $n^{-s/d}$: choosing a kernel is choosing how smooth you believe f is. The study's GPs use $\nu = \frac{5}{2}$: smooth enough for gradients, not so smooth that every wiggle must be analytic.

The frequency view (Bochner's theorem). A stationary kernel is valid exactly when it is the Fourier transform of a nonnegative function $S(\omega)$, its *spectral density*: $k(\tau) = \int S(\omega)\, e^{i\omega^{\mathsf{T}}\tau}\, d\omega$. The spectral density says how much variance sits at each frequency. SE has a Gaussian spectrum, which decays faster than any power: high frequencies are essentially forbidden, hence

infinite smoothness. Matérn-ν has $S(\omega) \propto (2\nu/\ell^2 + \|\omega\|^2)^{-(\nu+d/2)}$, a power-law tail: some high-frequency content survives, hence finite smoothness. Reading a kernel as a filter on frequencies is the natural view for anyone who has done spectral analysis of flow fields.

Connection (Random Fourier features and the study's `gp_draw` problems)**.** Bochner gives a cheap way to sample an approximate GP: draw frequencies ω_k from S, phases b_k uniform, and set $f(x) = \sqrt{2/D}\sum_{k=1}^{D} a_k \cos(\omega_k^\mathsf{T} x + b_k)$ with $a_k \sim \mathcal{N}(0,1)$. This is a finite-feature model of Chapter 1 whose kernel tends to k as D grows. The study's `gp_draw` test functions are built exactly this way (Student-t frequencies, i.e. a Matérn-5/2-like spectrum, $D = 256$).

2.3 Prediction: conditioning on the data

Take a GP prior $f \sim \mathcal{GP}(0, k)$ (a nonzero mean is handled in Section 2.4) and noisy observations $y_i = f(x_i) + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma_n^2)$ independent. At a new point x_*, the pair $(f(x_*), y)$ is jointly Gaussian with

$$\begin{pmatrix} f(x_*) \\ y \end{pmatrix} \sim \mathcal{N}\left(0,\ \begin{pmatrix} k(x_*, x_*) & k_*^\mathsf{T} \\ k_* & K + \sigma_n^2 I \end{pmatrix}\right), \qquad K_{ij} = k(x_i, x_j),\ (k_*)_i = k(x_i, x_*). \tag{2.6}$$

Conditioning (Result 1.15) gives the posterior at once:

$$\boxed{\mu(x_*) = k_*^\mathsf{T}(K + \sigma_n^2 I)^{-1} y, \qquad s^2(x_*) = k(x_*, x_*) - k_*^\mathsf{T}(K + \sigma_n^2 I)^{-1} k_*.} \tag{2.7}$$

Reading the mean. With $\alpha = (K+\sigma_n^2 I)^{-1} y$, the mean is $\mu(x) = \sum_{i=1}^{n} \alpha_i\, k(x_i, x)$: a weighted sum of n bumps, one centred on each data point, shaped like the kernel. The surrogate is as smooth as the kernel, and its gradient is available analytically, $\nabla\mu(x) = \sum_i \alpha_i \nabla_x k(x_i, x)$. This is why the study measured excellent gradients for the GP family: the surrogate is a smooth sum of smooth bumps.

Reading the variance. The variance starts at the prior $k(x_*, x_*)$ and subtracts what the data explain. Near data it shrinks; far away it returns to the prior. Two consequences:

- With $\sigma_n = 0$ the mean interpolates ($\mu(x_i) = y_i$) and the variance is zero at the data. With noise, it smooths, and the variance at a data point is below σ_n^2 but not zero.
- **The variance does not depend on y.** It depends only on where you sampled. A sampling rule that adds the point of maximum variance therefore ignores what f looked like there: it is a space-filling design in disguise. That is the mechanism behind the thesis-review finding (B-02) that the GP-variance scout performed about like a model-free maximin design. Only the hyper-parameters, refitted on the new data, carry information about f into such a rule.

Computing it. Never form an inverse. With one Cholesky factorisation:

1. $L = \mathrm{chol}(K + \sigma_n^2 I)$	$n^3/3$ operations, n^2 memory
2. $\alpha = L^\mathsf{T} \backslash (L \backslash y)$	two triangular solves, $2n^2$
3. mean $\mu(x_*) = k_*^\mathsf{T}\alpha$	n per test point
4. $v = L \backslash k_*$, variance $s^2(x_*) = k(x_*, x_*) - v^\mathsf{T} v$	n^2 per test point
5. $\log p(y) = -\frac{1}{2} y^\mathsf{T}\alpha - \sum_i \log L_{ii} - \frac{n}{2}\log 2\pi$	free (Section 2.6)

The n^3 step is why exact GPs stop at a few thousand points, and why the study caps the GP family at $n = 1000$. Sparse GPs (later chapter) replace K by a rank-m approximation, reducing the cost to nm^2.

2.4 Three names for one object

Kernel ridge regression. Forget probability. Among all functions in a space $\mathcal{H}$, find

$$\hat{f} = \arg\min_{g \in \mathcal{H}} \sum_{i=1}^{n} \big(y_i - g(x_i)\big)^2 + \lambda \|g\|_{\mathcal{H}}^2. \tag{2.8}$$

For the right space (below) the answer is the GP mean (2.7) with $\lambda = \sigma_n^2$. This is the infinite-dimensional version of Chapter 1's "MAP is ridge".

The space that makes it work. The *reproducing kernel Hilbert space* (RKHS) of k is the space of functions $g = \sum_j c_j\, k(z_j, \cdot)$ (and limits of such sums), with inner product $\langle k(x,\cdot),\, k(x',\cdot)\rangle_{\mathcal{H}} = k(x,x')$. The defining *reproducing property* is

$$g(x) = \langle g,\, k(x,\cdot)\rangle_{\mathcal{H}} \qquad \text{for every } g \in \mathcal{H}: \tag{2.9}$$

evaluating a function at x is taking its inner product with the kernel bump at x. The norm $\|g\|_{\mathcal{H}}$ measures roughness in the kernel's own terms; for Matérn kernels it is equivalent to a Sobolev norm (an integral of squared derivatives up to order $\nu + d/2$), the same kind of "energy" as in a variational formulation of a PDE.

Result 2.2 (Representer theorem)**.** The minimiser of (2.8) has the form $\hat{f}(x) = \sum_{i=1}^{n} \alpha_i\, k(x_i, x)$: only the bumps at the data are ever needed, even though $\mathcal{H}$ is infinite-dimensional.

Intuition. Projection again. Split any candidate $g = g_\parallel + g_\perp$, with $g_\parallel$ in the span of the data bumps $k(x_i,\cdot)$ and $g_\perp$ orthogonal to all of them. By the reproducing property $g_\perp(x_i) = \langle g_\perp,\, k(x_i,\cdot)\rangle = 0$, so $g_\perp$ is invisible at the data: the fit term is unchanged. But $\|g\|^2 = \|g_\parallel\|^2 + \|g_\perp\|^2$ (Pythagoras), so $g_\perp$ only adds penalty. The best choice is $g_\perp = 0$.

Radial basis function interpolation. RBF interpolation, from approximation theory and engineering, writes $s(x) = \sum_i c_i\, \varphi(\|x - x_i\|)$ and solves for c so that $s(x_i) = y_i$. With φ a PD kernel this is the noise-free GP mean ($\sigma_n = 0$). Some popular radial functions (multiquadric, thin-plate spline $r^2 \log r$) are only *conditionally* PD, which is repaired by adding a low-degree polynomial and orthogonality conditions; the thin-plate spline minimises a bending energy, its RKHS-type seminorm.

Kriging. Geostatistics (Krige, Matheron) derived the same predictor as the best linear unbiased predictor of a spatial field. *Simple kriging* assumes a known mean (our zero-mean GP). *Ordinary kriging* assumes an unknown constant mean β and estimates it by generalised least squares, $\hat{\beta} = (\mathbf{1}^\mathsf{T} A^{-1} y)/(\mathbf{1}^\mathsf{T} A^{-1}\mathbf{1})$ with $A = K + \sigma_n^2 I$, then kriges the residual. *Universal kriging* replaces the constant by a regression on basis functions. "Kriging" in engineering software (DACE, SMT) is a GP with one of these means.

Intuition. Kriging, GP regression, kernel ridge regression and RBF interpolation produce the same prediction for the same kernel and noise level. They differ in what else they give you and in how they choose hyper-parameters: the GP gives a variance and a likelihood for choosing ℓ, KRR usually picks λ and ℓ by cross-validation, RBF practice often fixes them by rule. In the study's results this is visible: `krr`, the RBFs and the GP family are neighbours in accuracy, and their differences come mostly from hyper-parameter choice.

2.5 What the error bar means

Result 2.3 (The GP variance is a worst-case error bound)**.** For noise-free data, the mean μ of (2.7) satisfies, for every f in the RKHS,

$$|f(x) - \mu(x)| \;\le\; s_0(x)\,\|f\|_{\mathcal{H}}, \tag{2.10}$$

where $s_0^2(x)$ is the noise-free posterior variance (called the *power function* in approximation theory).

Intuition. The interpolant is linear in the data: $\mu(x) = \sum_i c_i(x) f(x_i)$ for weights $c(x) = K^{-1}k_*$. By the reproducing property, $f(x) - \mu(x) = \langle f,\, k(x,\cdot) - \sum_i c_i(x) k(x_i,\cdot)\rangle_{\mathcal{H}}$, and Cauchy–Schwarz bounds this by $\|f\|_{\mathcal{H}}$ times the norm of the second factor. Expanding that norm with $\langle k(a,\cdot),\, k(b,\cdot)\rangle = k(a,b)$ gives exactly $k(x,x) - k_*^{\mathsf{T}} K^{-1} k_*$. So the GP's error bar is not only a Bayesian belief; it is the largest possible error over all functions of a given RKHS norm, i.e. of a given roughness.

Rates. For Matérn-ν kernels the power function shrinks like h^{ν}, where the *fill distance* h is the radius of the largest empty ball among the samples. A good space-filling design has $h \approx n^{-1/d}$, so the error falls like $n^{-\nu/d}$: Chapter 1's rate with $s = \nu$, now as a theorem. Two caveats the study sees in practice: the bound needs f to be in the RKHS (a discontinuous f is not, and the GP converges slowly near jumps), and it needs the right ℓ, which brings us to the hard part.

2.6 Choosing hyper-parameters: the marginal likelihood

The kernel has hyper-parameters $\theta = (\sigma_f, \ell_1, \ldots, \ell_d, \sigma_n)$. Integrating f out (a linear-Gaussian computation) gives the probability of the data under θ, the *marginal likelihood* or *evidence*: $y \mid \theta \sim \mathcal{N}(0, A_\theta)$ with $A_\theta = K_\theta + \sigma_n^2 I$, so

$$\log p(y \mid \theta) = \underbrace{-\tfrac{1}{2}\, y^{\mathsf{T}} A_\theta^{-1} y}_{\text{data fit}} \;\; \underbrace{-\,\tfrac{1}{2}\log\det A_\theta}_{\text{complexity penalty}} \;-\; \tfrac{n}{2}\log 2\pi. \tag{2.11}$$

Maximising it over θ is *type-II maximum likelihood* (ML-II), the default in every GP package.

Intuition (Automatic Occam's razor)**.** The first term rewards explaining the data. The second term is a penalty: $\det A_\theta$ measures the volume of data sets the model considers plausible, so a very flexible model (short ℓ, large σ_f) spreads its probability over many possible data sets and assigns less to the one observed. A model that is too rigid cannot explain the data; one that is too flexible could explain anything and is therefore unconvincing about this data set. The evidence peaks in between. Its gradient is available in closed form, $\partial_\theta \log p = \tfrac{1}{2}\alpha^{\mathsf{T}}(\partial_\theta A)\alpha - \tfrac{1}{2}\operatorname{tr}(A^{-1}\partial_\theta A)$, and L-BFGS does the rest.

2.6.1 Where ML-II goes wrong

The evidence is not concave in θ, and with few points it can prefer nonsense. Both failures appeared in the study, so here they are explicitly.

Several local optima. Figure 2.2 shows the evidence (with σ_f optimised out) for 40 noisy samples of $\sin(2\pi x) + 0.3\sin(20\pi x)$. There are two peaks. One, at $\ell = 0.051$ and noise/signal ratio 0.003 (log evidence -9.2), fits both components. The other, at $\ell = 0.22$ and ratio 0.087 (-17.5), explains the slow component and calls the fast one noise. Both are reasonable stories; a gradient optimizer started near the second never finds the first. This is the "signal fitted as noise" failure of bug N-14.

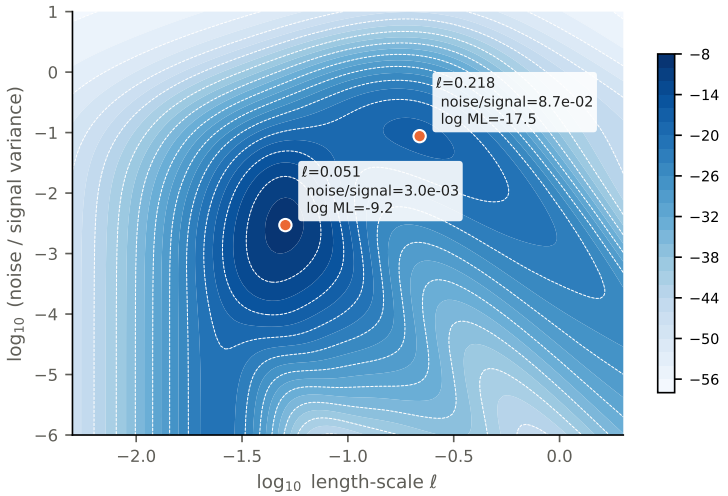


Figure 2.2: Log marginal likelihood over length-scale and noise ratio for a slow-plus-fast signal. Two local maxima (orange): "fit everything" and "fast part is noise". Which one an optimizer reaches depends on where it starts.

Too few points: the length-scale collapses. Figure 2.3 fits a GP to six points of the Forrester function. With ℓ fixed at a sensible 0.15, the fit is good (NRMSE 0.125). Letting ML-II choose, with ten restarts, drives ℓ to its lower bound 0.01: the posterior mean becomes a row of spikes at the data and the prior mean everywhere else (NRMSE 0.94). And the evidence genuinely prefers it: -8.51 against -9.24. With six points, "independent values at each sample" is a perfectly good explanation of six numbers, and it costs less complexity than a smooth function that must bend through all of them. The data cannot rule it out.

Ill-conditioning. For 50 evenly spaced points on $[0, 1]$, the condition number of K:

length-scale ℓ	SE	Matérn 5/2	Matérn 1/2
0.02	5.6×10^{1}	1.2×10^{1}	4.5
0.05	1.1×10^{12}	1.0×10^{3}	24
0.1	1.6×10^{18}	5.4×10^{4}	91
0.2	1.3×10^{19}	3.0×10^{6}	320
0.5	1.4×10^{19}	5.1×10^{8}	1.4×10^{3}

When samples are close relative to ℓ, neighbouring columns of K are nearly identical (the monomial problem of Chapter 1 again). SE is the worst because its spectrum dies fastest: its eigenvalues λ_j decay super-exponentially, so K is numerically singular beyond $\ell \approx 0.05$ here. Adding a *nugget* (a small σ_n^2, or "jitter") shifts every eigenvalue up and is exactly Chapter 1's ridge filter. It is one reason the study prefers Matérn 5/2 to SE, and why every GP in the study has a nugget floor.

What the study found, and the remedies. In E3 between 2 and 9% of GP-family fits (1 restart) were more than twice as bad as a kernel ridge regression on the same data, the signature

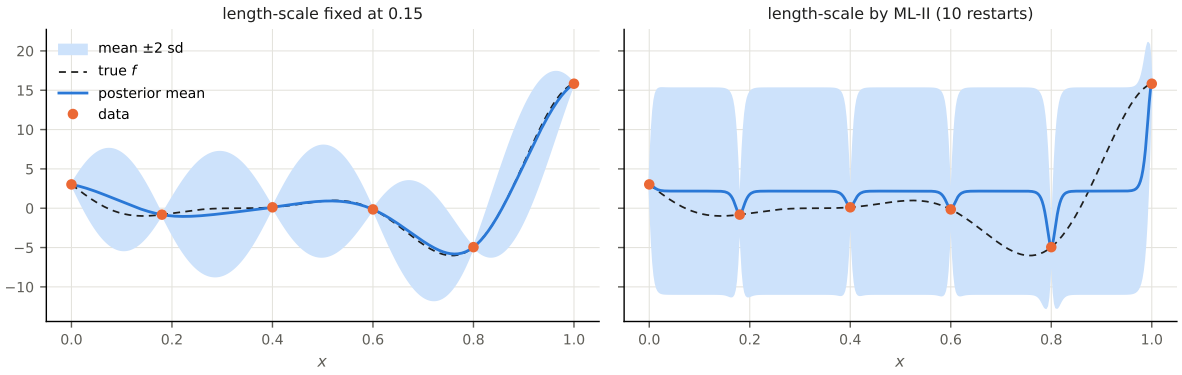


Figure 2.3: GP fits to six samples of the Forrester function. Left: length-scale fixed at 0.15. Right: length-scale chosen by ML-II, which collapses to its bound (0.01) and returns the prior between samples. Note how the variance band (shaded) reflects only the spacing of the samples, not the values.

of a bad optimum, and five restarts fixed most of them. Standard remedies, in increasing cost: several restarts; sensible bounds (a length-scale floor near the smallest sample spacing rules out the collapse above); a prior on θ and maximising the posterior instead of the likelihood ("MAP-II"); choosing θ by cross-validation; and, most robust, averaging over θ with Markov-chain Monte Carlo instead of choosing one value. Kernel Flows, used by one of the study's learners, is a cross-validation-like criterion built for exactly this problem.

2.7 When a GP is the wrong model

Property of f	Why a stationary GP struggles	Variant (later chapters)
Jumps, kinks	not in the RKHS; one global ℓ must be short everywhere, so oscillation (Gibbs-like) near the jump	treed GP, local GP, partitioned models
Nonstationary (rough here, smooth there)	one length-scale for the whole domain	warped GP, deep GP, Kernel Flows
Noise that varies or has outliers	Gaussian likelihood, constant σ_n	heteroscedastic GP, Student-t likelihood
High dimension	distances concentrate; d length-scales to fit	ARD, active-subspace GP, KPLS
Large n	n^3 cost, n^2 memory	sparse GP, local GP

The study's E3 results line up with this table: GP variants win smooth, ridge, multiscale and engineering problems (where f is in the RKHS of a Matérn kernel with sensible length-scales), and lose discontinuities and steep drops to trees.

Notes

- Derivatives of a GP are again a GP (differentiation is linear), with covariance $\partial_x \partial_{x'} k$. Gradient observations can therefore be conditioned on exactly like values: gradient-enhanced kriging, the subject of the planned gradient study (E13).

- Multi-output and multi-fidelity GPs (co-kriging) use a kernel on (input, output index); a later chapter.

Summary of the formulas to keep

- GP posterior: $\mu = k_*^\mathsf{T}(K + \sigma_n^2 I)^{-1} y$, $s^2 = k_{**} - k_*^\mathsf{T}(K + \sigma_n^2 I)^{-1} k_*$: Gaussian conditioning, computed with one Cholesky factorisation.
- The mean is $\sum_i \alpha_i k(x_i, \cdot)$ (representer theorem); the variance ignores y.
- Kriging = GP mean = kernel ridge regression ($\lambda = \sigma_n^2$) = RBF interpolation ($\sigma_n = 0$).
- Matérn ν = smoothness s; error $\leq s_0(x)\|f\|_{\mathcal{H}}$ and $\approx n^{-\nu/d}$.
- Evidence = data fit − complexity; non-concave, can collapse with few points; restarts, bounds, priors, nugget.